

Synthèse : ReAct, l'IA et la confrontation au terrain

Contexte et Problématique

Le Pattern ReAct : Définition et Principes

Qu'est-ce que ReAct ?

ReAct (Reasoning and Acting) est un pattern d'interaction pour les agents IA qui combine : 1. Raisonement (Reasoning) : L'agent génère des hypothèses, des plans ou des solutions en explicitant sa logique (via des traces ou chain-of-thought). 2. Action (Acting) : L'agent teste ses hypothèses dans un environnement contrôlé (simulation, exécution de code, appels à des outils externes). 3. Feedback : L'agent reçoit des résultats concrets (succès, échec, données terrain) et ajuste son raisonnement en conséquence. → Boucle vertueuse : Raisonement → Action → Feedback → Nouveau Raisonement → ...

Application de ReAct à ton Projet : Essaim de Drones

Cas d'usage : Gestion de collision pour 10 drones autonomes

Objectif : Implémenter un système robuste en Rust, avec une latence < 100ms.

Étape 1 : Initialisation

- Agent : Claude Code (génération de code) + Gazebo (simulation). - Prompt : Comment implémenter un système de gestion de collision pour 10 drones autonomes en Rust, avec une latence < 100ms ?

Étape 2 : Première Itération ReAct

1. Raisonnement : - Un essaim de drones nécessite un protocole de communication décentralisé. - Option 1 : Algorithme basé sur des règles locales (évitement par capteurs) → Simple et rapide. - Option 2 : Consensus distribué (ex : Paxos) → Robuste mais complexe. - Choix : Tester l'option 1 en premier. 2. Action : Génération du code Rust pour l'évitement par capteurs. 3. Feedback : Test en simulation (Gazebo) : Échec : 2 drones entrent en collision si leurs capteurs ont une latence > 50ms.

Étape 3 : Deuxième Itération ReAct

1. Raisonnement : Le problème vient de la latence des capteurs. Il faut ajouter une marge de sécurité ou changer d'algorithme. 2. Action : Modification du code pour ajouter une marge de 20% sur la distance de sécurité. 3. Feedback : Succès : Plus de collision, mais la latence moyenne passe à 120ms (dépassement du seuil).

Étape 4 : Troisième Itération ReAct

1. Raisonnement : La marge de sécurité augmente la latence. Essayons un algorithme de consensus distribué (leader-follower). 2. Action : Implémentation d'un leader-follower en Rust. 3. Feedback : Succès : Latence < 100ms, mais le leader est un point de défaillance unique.

Étape 5 : Quatrième Itération ReAct

1. Raisonnement : Il faut un algorithme sans leader. Essayons un modèle inspiré des Boids (swarm intelligence). 2. Action : Implémentation d'un algorithme de type Boids. 3. Feedback : Succès : Latence < 100ms, pas de collision, pas de point de défaillance unique. Par contre, la consommation CPU est élevée.

Bonnes Pratiques pour Utiliser ReAct

1. Structurer les Prompts

Utilise des prompts ReAct pour guider l'agent : `` Tu es un ingénieur système expert en robotique et Rust. Pour chaque problème, suis ce processus : 1. [RAISONNEMENT] : Explique ta logique étape par étape. 2. [ACTION] : Propose une solution concrète (code, algorithme, architecture). 3. [TEST] : Décrit comment tester cette solution (simulation, tests unitaires, etc.). 4. [FEEDBACK] : Attends les résultats des tests, puis ajuste ton raisonnement. Problème : Comment implémenter un système de gestion de collision pour 10 drones autonomes en Rust, avec une latence < 100ms ? ``

2. Automatiser les Boucles

- Utilise LangChain ou AutoGen pour chaîner les actions (génération → test → analyse).

3. Documenter les Itérations

- Garde une trace de chaque boucle ReAct (raisonnement → action → feedback).

Ressources pour Aller Plus Loin

Papers

- [ReAct: Synergizing Reasoning and Acting in Language Models](<https://arxiv.org/abs/2210.03629>) (Yao et al., 2022) - [AgentVerse: Facilitating Multi-Agent Collaboration](<https://arxiv.org/abs/2308.10848>) (Microsoft) - [SWE-agent: Agent for Software Engineering](<https://arxiv.org/abs/2308.08226>)

Synthèse des Échanges avec Xavier

1. Le Problème de Base

- L'IA génère du plausible, pas du juste. - Exemple : Un code ou une analyse peut paraître correct(e) mais échouer en pratique.

2. La Solution : ReAct

- ReAct = Raisonnement + Action + Feedback. - Application : L'agent propose une solution, la teste en simulation, et ajuste son raisonnement.

3. Cas Pratique : Essaim de Drones

- Itération 1 : Algorithme d'évitement par capteurs → Échec (collisions dues à la latence). - Itération 2 : Ajout d'une marge de sécurité → Échec (latence trop élevée). - Itération 3 : Algorithme leader-follower → Succès partiel (latence OK, mais point de défaillance unique). - Itération 4 : Algorithme inspiré des Boids → Succès (latence OK, pas de collision, pas de point de défaillance). - Validation humaine : Ajustement final et déploiement.

4. Outils Clés

- Orchestration : LangChain, AutoGen. - Simulation : Gazebo, PyBullet. - Validation : Clippy, pytest, SonarQube.